

NAME

comm – compare two sorted files line-by-line (fail-safe version)

SYNOPSIS

comm <file1> <file2>

DESCRIPTION

A robust, protective wrapper for the standard **comm** utility.

The native system **comm** requires both input files to be lexically sorted prior to execution. If unsorted inputs are provided, the system binary fails silently, emitting incomplete or corrupted matching data without warning.

This refined version runs defensive checks on the inputs using **sort -C**. If either file is found to be unsorted, it leverages shell process substitution to sort the streams dynamically on the fly, guaranteeing accurate comparison results while leaving the original files on the disk untouched.

THE SCRIPT

This manual entry documents the following standalone intercept script:

```
#!/bin/bash
if [[ $# -ne 2 ]]; then
    echo "Usage: comm <file1> <file2>"
    exit 1
fi
FILE1="$1"
FILE2="$2"

sort -C "$FILE1" 2>/dev/null; SORTED1=$?
sort -C "$FILE2" 2>/dev/null; SORTED2=$?

if [[ $SORTED1 -eq 0 && $SORTED2 -eq 0 ]]; then
    /usr/bin/comm "$FILE1" "$FILE2"
elif [[ $SORTED1 -ne 0 && $SORTED2 -eq 0 ]]; then
    /usr/bin/comm <(sort "$FILE1") "$FILE2"
elif [[ $SORTED1 -eq 0 && $SORTED2 -ne 0 ]]; then
    /usr/bin/comm "$FILE1" <(sort "$FILE2")
else
    /usr/bin/comm <(sort "$FILE1") <(sort "$FILE2")
fi
```

TECHNICAL NOTES**Defensive Checking:**

The script uses **sort -C** to audit the files. This check is fast and computationally inexpensive, ensuring that already-sorted files suffer zero performance penalty.

Memory Efficiency:

By using bash process substitution <(), sorted data streams are piped directly into the **comm** execution block via anonymous named pipes. This avoids the friction of creating, tracking, and clearing out temporary files on the disk.

DIAGNOSTICS

Returns the exit status of the underlying **comm** command, or 1 if the incorrect number of file arguments is passed.

AUTHOR

William Orian Wear (Bill Wear)